Daily Coding Assignment Wipro NGA .NET Cohort

Assignment 1: Advanced Razor Pages Implementation

Objective: Develop a Razor Pages application with a focus on model binding, partial views, and routing. This assignment will deepen your understanding of Razor Pages by implementing complex model binding, using partial views, and customizing routing.

Problem Statement

Create a Razor Pages application that includes functionality for handling complex model types and collections, using partial views for reusability, and customizing routes to manage data effectively.

User Stories and Expectations

User Story 1: Model Binding with Complex Types and Collections

- **As a developer**, you need to create a page that binds to a complex model type and handles collections of data.
- **Acceptance Criteria**:
  - Create a `Product` model with properties such as `ProductID`, `Name`, `Description`, and a list of `Category` objects.
  - Develop a Razor Page that allows users to add new products with associated categories and display a list of products with their categories.

User Story 2: Implement Partial Views

- **As a developer**, you need to use partial views to make your Razor Pages more modular and reusable.
- **Acceptance Criteria**:
  - Create a partial view to display a product summary (e.g., name, description) and include this partial view in the main Razor Page that lists products.
  - Ensure that the partial view is correctly rendered for each product on the main page.

User Story 3: Customize Routing

- **As a developer**, you need to configure custom routes and route parameters to manage data efficiently.
- **Acceptance Criteria**:
  - Implement custom routes to access product details and categories using route parameters (e.g., `/Products/{id}`).
  - Configure routing in the `Startup.cs` file to handle these custom routes and ensure they correctly map to the Razor Pages.

Assignment Outcome

By completing this assignment, you will gain practical experience with advanced Razor Pages features, including handling complex model binding, using partial views for code reusability, and customizing routing.

Best Practices for Submission:

1. **Code Structure**: Organize your Razor Pages and partial views logically. Ensure that your model classes and page files are well-structured.
2. **Documentation**: Provide comments to explain the purpose of the model binding and routing configurations.
3. **Testing**: Verify that all routes function as expected, partial views are correctly rendered, and model binding works for complex types and collections.

Assignment 2: MVC Pattern and Model Binding

Objective: Create an MVC application that demonstrates an understanding of the MVC pattern, including setting up an MVC project, handling model binding for both simple and complex types, and configuring views and controllers.

Problem Statement

Build an MVC application that includes a basic implementation of the MVC pattern. The application should demonstrate model binding with simple and complex types and manage data through controllers and views.

User Stories and Expectations

User Story 1: Set Up an MVC Project

- **As a developer**, you need to set up a new MVC project and understand its structure.
- **Acceptance Criteria**:
    - Create a new MVC project using Visual Studio or the .NET CLI.
    - Ensure that the project includes basic folders for Models, Views, and Controllers.

User Story 2: Model Binding with Simple Types

- **As a developer**, you need to implement model binding for simple types (e.g., strings, integers) in a form submission.
- **Acceptance Criteria**:
    - Create a model class with properties like `FirstName`, `LastName`, and `Age`.

    - Develop a view with a form to input these properties and a controller action to handle the form submission and display the submitted data.

User Story 3: Model Binding with Complex Types

- **As a developer**, you need to implement model binding for complex types (e.g., nested models) in a form submission.
- **Acceptance Criteria**:
    - Extend the model from User Story 2 to include a nested model (e.g., an `Address` model with properties like `Street`, `City`, and `ZipCode`).
    - Create a view and form to input address details along with user details and handle the form submission in the controller.

User Story 4: Understanding MVC Components

- **As a developer**, you need to understand and demonstrate the roles of Models, Views, and Controllers in the MVC pattern.
- **Acceptance Criteria**:
    - Clearly separate the roles of models, views, and controllers in your application.
    - Ensure that the controller manages user input and interaction, the model represents the data, and the view presents the data to the user.

Assignment Outcome

By completing this assignment, you will gain a deeper understanding of the MVC pattern, including how to set up an MVC project, handle model binding for different types, and properly structure MVC components.

Best Practices for Submission:

1. **Code Organization**: Ensure your MVC project is well-organized with separate folders for Models, Views, and Controllers.
2. **Documentation**: Include comments to explain the use of model binding and the MVC pattern components.
3. **Testing**: Test the form submissions to ensure model binding works for both simple and complex types and that MVC components are correctly interacting.

These assignments will provide a solid foundation in Razor Pages and MVC development, offering practical experience with key concepts and best practices in ASP.NET Core.